

Advanced Time-Stepping Methods: Spectral Deferred Correction Scheme

DS-289 : Numerical Solution of Differential Equations

Abinav Thangaraju Sethupathy, BTech Mathematics & Computing
Vinothini S, PhD Computational & Data Sciences

Wednesday 8th April, 2026

Table of Contents

1 Introduction

- Why Spectral Deferred Correction?
- Methodology

2 Experiments

- ODE Experiments
- PDE Experiments

3 Future Roadmap

Why Spectral Deferred Correction?

- 1 **Iterative Scalability of Accuracy:** One of the biggest limitations of traditional methods like RK4 is that their accuracy is fixed — we get 4th order, and that's it. SDC takes a different approach. By simply running additional correction sweeps, we can keep improving the solution's precision without starting from scratch or deriving new coefficients. It's a much more practical way to reach high accuracy.
- 2 **Mitigation of the Runge Phenomenon:** SDC avoids this by using spectral nodes — like Gauss-Lobatto or Chebyshev points — which naturally bunch up near the boundaries rather than spreading out evenly. That simple change in how points are distributed is enough to keep the oscillations from appearing in the first place.
- 3 **Flexibility in Multi-Physics Coupling:** Its structure makes it easy to treat different parts of a problem differently — for example, handling stiff reactions implicitly while dealing with non-stiff advection explicitly — all within the same iterative loop. Traditional schemes often struggle here, but SDC handles it in a clean, modular way.

Methodology

Consider the initial value problem

$$y'(t) = f(t, y(t)), \quad y(t_0) = y_0$$

Convert into integral form of the ODE

$$y(t) = y_n + \int_{t_n}^t f(s, y(s)) ds$$

Choose collocation nodes (Equally spaced or Unequally spaced)

$$t_n = t_0 < t_1 < t_2 \cdots < t_M = t_{n+1}$$

Polynomial interpolation of f - Lagrange can be used for equally and unequally spaced

$$f(s, y(s)) \approx \sum_{j=1}^M f(t_j, Y_j) l_j(s) \quad \text{where } l_j(s) = \prod_{k=1, k \neq j}^M \frac{s - t_k}{t_j - t_k}$$

Methodology

Divided difference can be used for equally spaced and unequally spaced

$$f(s, y(s)) \approx a_1 + a_2(s - t_1) + a_3(s - t_1)(s - t_2) + \cdots + a_M \prod_{j=1}^{M-1} (s - t_j)$$

Quadrature approximation

$$\int_{t_n}^{t_i} f(s, y(s)) ds \approx \int_{t_n}^{t_i} \sum_{j=1}^M f(t_j, Y_j) l_j(s) ds \approx \sum_{j=1}^M f(t_j, Y_j) \int_{t_n}^{t_i} l_j(s) ds \approx \sum_{j=1}^M Q_{ij} f(t_j, Y_j)$$

where $Q_{ij} = \int_{t_n}^{t_i} l_j(s) ds$ collocation equation

$$Y_i \approx y_n + \sum_{j=1}^M Q_{ij} f(t_j, Y_j), Y = y_n \mathbf{1} + QF(Y)$$

Methodology

Residual (Defect)

$$R(Y) = y_n 1 + QF(Y) - Y$$

Picard Iteration

$$y^{k+1}(t) = y_n + \int_{t_n}^t f(s, y^k(s)) ds, Y^{k+1} = y_n 1 + QF(Y^k)$$

SDC Correction Sweep

$$Y_{m+1}^{k+1} = (Y_m^{k+1} + \Delta t_m [f(t_{m+1}, Y_{m+1}^{k+1})]) + \left(\sum_{j=1}^M (Q_{m+1,j} - Q_{m,j}) f(t_j, Y_j^k) - \Delta t_m [f(t_{m+1}, Y_{m+1}^k)] \right)$$

$$Y_{m+1}^{k+1} = Y_m^{k+1} + \Delta t_m [f(t_{m+1}, Y_{m+1}^{k+1}) - f(t_{m+1}, Y_{m+1}^k)] + \sum_{j=1}^M (Q_{m+1,j} - Q_{m,j}) f(t_j, Y_j^k)$$

The experiments performed till now are:

① ODE Experiments

- ① Exp 1.1: Explicit SDC on basic ODE
- ② Exp 1.2: Stability Comparison on a Stiff ODE
- ③ Exp 1.3: Efficiency and Accuracy Comparison (Stiff ODE)
- ④ Exp 1.4: Verifying the Accuracy Bottleneck Formula

② PDE Experiments

- ① Exp 2.1: The Linear Heat Equation and Error Comparison
- ② Exp 2.2: The Linear Advection Equation - Lucky Correction
- ③ Exp 2.3: The Linear Advection Equation - Unstable Explicit Euler
- ④ Exp 2.4: The Linear Advection Equation - SDC Supremacy
- ⑤ Exp 2.5: The Non-linear Diffusion Equation - Capturing Transient Phase
- ⑥ Exp 2.6: The Viscous Burgers' Equation - IMEX Splitting

Experiment 1.1: Explicit SDC on basic ODE

Aim: To verify higher-order accuracy achieved by SDC using Explicit Euler.

Setup:

① ODE:

$$u'(t) = -u(t), \quad u(0) = 1, \quad t \in [0, 1]$$

② Quadrature: Radau-Right (Gauss-Legendre nodes with one node on the right end)

③ Basic Approximator: Explicit Euler

④ No. of nodes: 5

⑤ No. of correction sweeps: 5

⑥ Step size $(\Delta t) = 0.1$

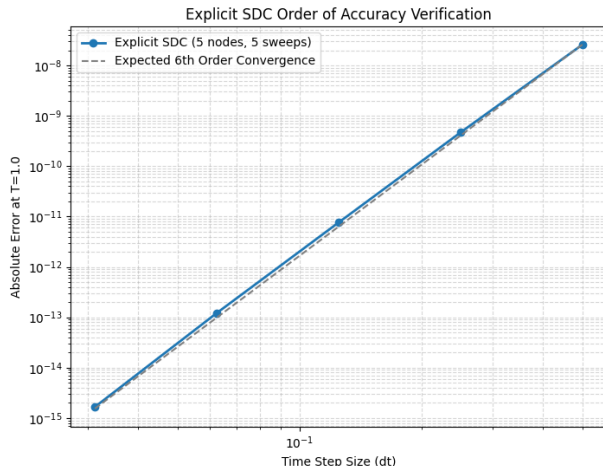
ODE Experiments

Results:

dt	Error
0.50000	2.59e-08
0.25000	4.67e-10
0.12500	7.63e-12
0.06250	1.22e-13
0.03125	1.67e-15

Conclusion:

6th order accuracy verified
(Order 1 Euler + 5 sweeps)



Experiment 1.2: Stability Comparison on a Stiff ODE

Aim: To demonstrate the necessity of Implicit SDC when solving stiff equations, and visualize the failure of Explicit sweeps.

Setup:

① ODE:

$$u'(t) = -1000u(t), \quad u(0) = 1, \quad t \in [0, 1]$$

② Quadrature: Radau-Right (Gauss-Legendre nodes with one node on the right end)

③ No. of nodes: 3

④ No. of correction sweeps: 3

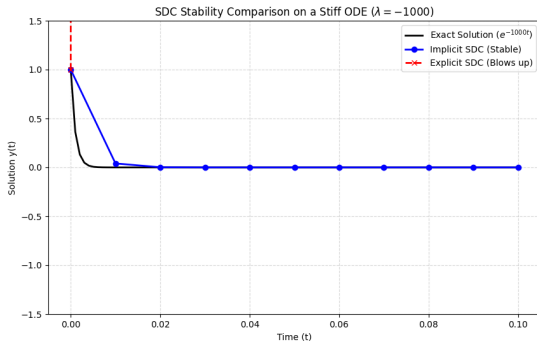
⑤ Step size $(\Delta t) = 0.01$

Methods Compared: Explicit SDC (Base: Exp-Euler) vs Implicit SDC (Base: Imp-Euler)

ODE Experiments

Results:

- 1 Explicit SDC: Underwent massive numerical oscillation and overflowed.
- 2 Implicit SDC: Remained absolutely stable, perfectly tracking the exact solution $e^{(-1000t)}$ down to zero.



Conclusion: Explicit sweepers are unconditionally unstable for highly stiff problems. Implicit SDC must be used for stiff components.

Experiment 1.3: Efficiency and Accuracy Comparison (Stiff ODE)

Aim: To prove that SDC is computationally more efficient than standard low-order implicit methods by comparing accuracy at a large, fixed step size.

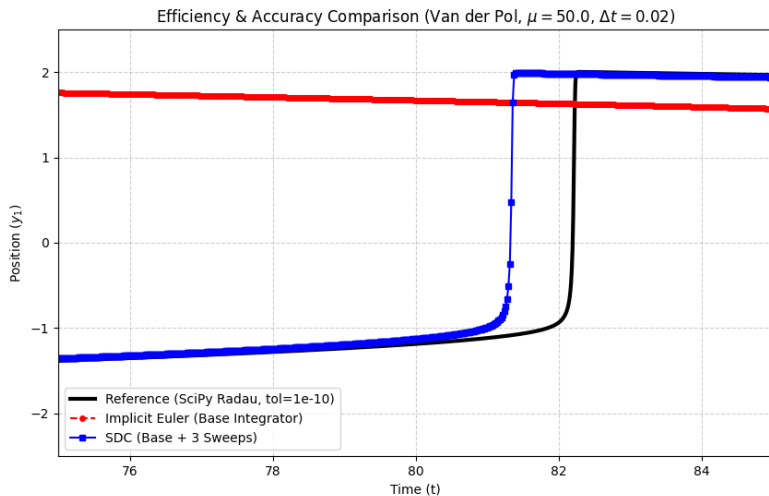
Setup: ODE: Van der Pol equation

$$\frac{d^2u}{dt^2} - 50(1 - u^2)\frac{du}{dt} + u = 0, \quad u(0) = 1, u'(0) = 0, \quad t \in [0, 100]$$

Methods Compared:

- 1 Implicit Euler (3 nodes, $\Delta t = 0.02$)
- 2 Implicit SDC (3 nodes, 3 correction sweeps, $\Delta t = 0.02$)
- 3 Reference - SciPy solve_ivp - Radau method

ODE Experiments



ODE Experiments

Comments:

- ① The second-order ODE has a non-linear term, so Newton-Raphson method is used to resolve it in one of the coupled first-order ODEs
- ② SDC allows for a larger time step, but Newton-Raphson is a limiting factor here
- ③ The ODE has no analytical solution; a SciPy solver with $1e^{-10}$ tolerance was used

Results:

- ① Simple Implicit Euler suffered severe numerical dissipation, completely missing the timing and shape of the oscillation's sharp drop.
- ② The SDC method, using the same large step size, achieved the shape of the reference solution. The slight shift in time was due to phase error caused by the solver's fixed nature

Conclusion: SDC allows massive time steps on stiff non-linear problems without sacrificing accuracy, while Implicit Euler would require drastically smaller time steps to do so.

Experiment 1.4: Verifying the Accuracy Bottleneck Formula

Aim: To verify the SDC's theoretical order of accuracy = $\min(\text{order of quadrature, order of base integrator} + \text{number of sweeps})$.

Setup:

① ODE:

$$u'(t) = -u(t), \quad u(0) = 1, \quad t \in [0, 1]$$

② Quadrature: Radau-Right (Gauss-Legendre nodes with one node on the right end)

③ Basic Approximator: Explicit Euler

④ No. of nodes: 3

⑤ Step size $(\Delta t) = 0.2$

⑥ Independent Variable: No. of correction sweeps $K = [1, 8]$

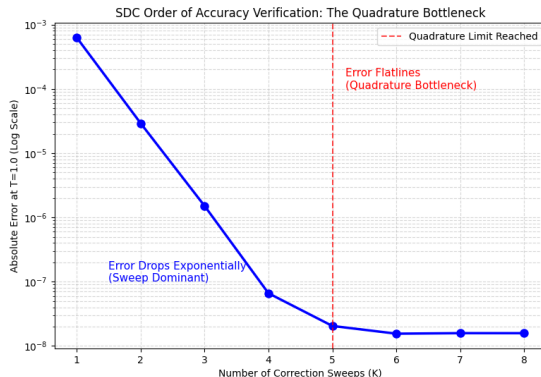
ODE Experiments

Results:

For $K = 1-4$, the absolute error dropped exponentially. At $K = 4$ (Base 1 + 4 Sweeps = 5th Order), the scheme hit the mathematical limit on order of accuracy. For $K = 5-8$, the error remained completely static.

Conclusion:

The experiment visually proves the theoretical order of accuracy of SDC. Performing more sweeps than the underlying quadrature can support is wasted computation.



The following PDEs were initially discretized on the spatial domain:

- ① Linear Heat Equation (Parabolic)
- ② Linear Advection Equation (Hyperbolic)
- ③ Non-linear Diffusion (Parabolic)
- ④ Viscous Burgers' Equation (Hyperbolic)

The obtained lumped system was then solved using Spectral Deferred Correction.

Consider the 1D Heat Equation:

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$$

Using a second-order central difference for the spatial derivative on a grid with spacing Δx :

$$\frac{du_i}{dt} = \frac{\alpha}{(\Delta x)^2} (u_{i-1} - 2u_i + u_{i+1})$$

This results in a system of ODEs:

$$\mathbf{u}'(t) = \mathbf{A}\mathbf{u}(t)$$

where $\mathbf{A}$ is the tridiagonal differentiation matrix representing the diffusion operator.

SDC can now be applied on this lumped system (Method of Lines).

Experiment 2.1: The Linear Heat Equation & Error Comparison

Aim: To solve the 1D Heat Equation using the Method of Lines and compare Implicit SDC against a standard Implicit Finite Difference scheme (BTCS).

Setup: PDE:

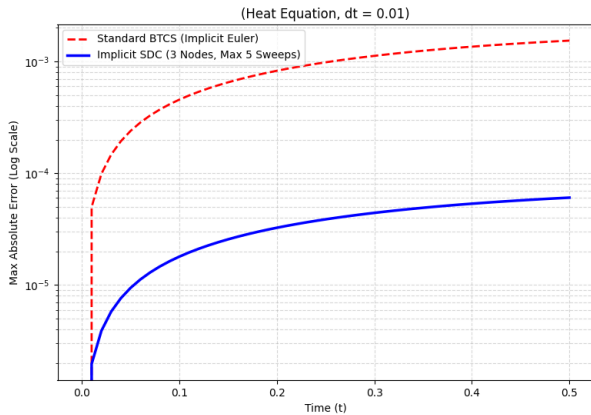
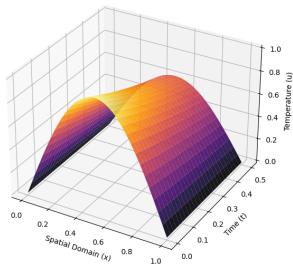
$$u_t = 0.1u_{xx}, \quad x \in [0, 1], \quad t \in [0, 0.5]$$

Methods Compared:

- ① Standard BTCS (Implicit Euler in time, Central Difference in space, $\Delta t = 0.01$)
- ② Method of Lines - Implicit SDC:
 - ① Spatial Discretization: 2nd order Central Difference
 - ② Quadrature: Radau-Right
 - ③ Basic Approximator: Implicit Euler
 - ④ No. of nodes: 3
 - ⑤ Time step: $\Delta t = 0.01$

PDE Experiments

1D Heat Equation solved with Implicit SDC (dt = 0.01)



Results:

- ① Explicit stability limit: $\Delta t = 0.0012$ - No hope of implementing this at $\Delta t = 0.01$
- ② Both implicit methods remained stable at $\Delta t = 0.01$.
- ③ Backward Time Central Space:
 - ① Average Error = 9.28×10^{-4}
 - ② Error at $T_{end} = 1.54 \times 10^{-3}$
- ④ Method of Lines - Implicit SDC:
 - ① Average Error = 3.65×10^{-5}
 - ② Error at $T_{end} = 6.05 \times 10^{-5}$

Conclusion: SDC solves the accuracy-stability tradeoff in parabolic PDEs. While BTCS guarantees stability, it yielded an average error of $\approx 10^{-3}$. SDC utilized the stable implicit base step but recovered high-order accuracy through sweeps, while using the exact same step size.

Experiments 2.2 – 2.4: The Linear Advection Equation

Aim: To demonstrate SDC's ability to eradicate numerical dissipation and phase error in hyperbolic PDEs, while proving its expanded stability and efficiency over base explicit methods.

Setup: PDE:

$$u_t + cu_x = 0, \quad x \in [0, 1], \quad t \in [0, 1.0]$$

Initial Condition: Gaussian Pulse (Periodic Boundaries)

Methods Compared:

- ① Standard Explicit Euler (Upwind spatial discretization, Δt varies)
- ② Method of Lines - Explicit SDC:
 - ① Spatial Discretization: 1st-Order Upwind & 3rd-Order Upwind
 - ② Quadrature: Radau-Right
 - ③ Basic Approximator: Explicit Euler
 - ④ No. of nodes: 3
 - ⑤ Time step: $\Delta t = 0.008$ (Massive) and $\Delta t = 0.001$ (Stable)

Results

① Exp 2.2: (1st-Order Space, $\Delta t = 0.008$)

- ① Both methods used a first-order upwind spatial discretization. Δt was chosen to be 0.008, obtained by fixing the CFL number at 0.8 times of the stability limit.
- ② Explicit Euler looked accurate due to a lucky cancellation between spatial smearing (positive diffusion) and temporal sharpening (negative diffusion).
- ③ SDC lowered the temporal error, so no cancellation occurred and the highly dissipative nature of the 1st-order spatial grid was exposed.

② Exp 2.3: (3rd-Order Space, $\Delta t = 0.008$)

- ① The spatial discretization was changed to 3rd order upwind, to prevent it from limiting the time schemes accuracy. Δt was kept the same as before.
- ② Increasing spatial accuracy pushed eigenvalues outside Explicit Euler's stability region, causing violent blow-up (amplitudes of ± 150).
- ③ SDC correction sweeps expanded the stability region, safely integrating the high-order space at the exact same Δt to perfectly resolve the wave.

Results:

❶ **Exp 2.4:** (3rd-Order Space, Variable Δt)

- ❶ To bring the Explicit Euler into its stability region, the time step was decreased only for it, leaving the Explicit SDC as such.
- ❷ Standard Explicit Euler ($\Delta t = 0.001$): Required 1000 evaluations to remain stable, yet still exhibited noticeable phase lag.
- ❸ Explicit SDC ($\Delta t = 0.008$): Required ≈ 1500 evaluations. Perfectly tracked the exact analytical solution with zero phase error.

Conclusion:

High-order spatial matrices are needed to prevent numerical diffusion, but they severely restrict explicit stability limits. For a $\sim 50\%$ increase in computational cost, SDC permits an $8\times$ larger time step while removing significant dissipation and dispersion effects. This is a major step-up over low-order explicit methods

PDE Experiments

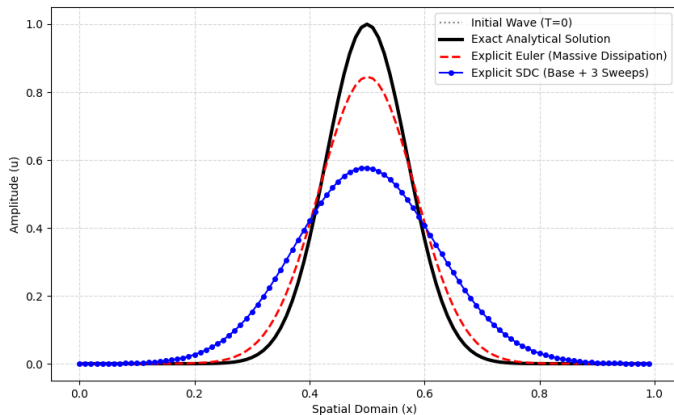


Figure: Exp 2.2: (1st-Order Space, $\Delta t = 0.008$)

PDE Experiments

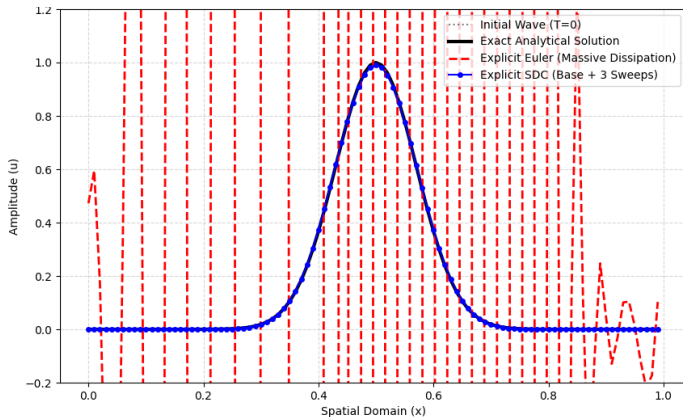


Figure: Exp 2.3: (3rd-Order Space, $\Delta t = 0.008$)

PDE Experiments

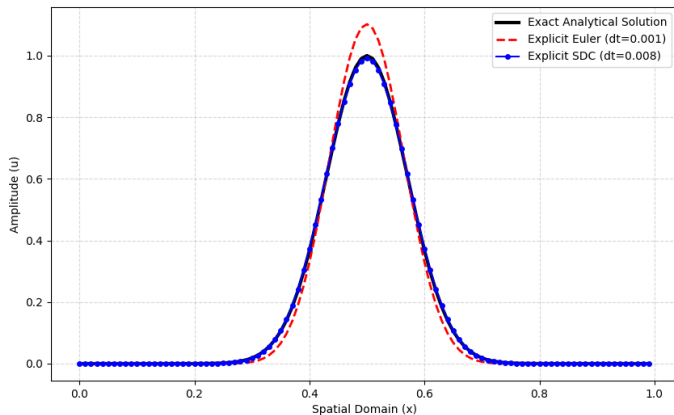


Figure: Exp 2.4: (3rd-Order Space, Variable Δt)

Experiment 2.5: Capturing the Transient Phase of Nonlinear Diffusion

Aim: To demonstrate how Spectral Deferred Corrections (SDC) reduce artificial numerical smearing in fully implicit time integration.

Setup:

- 1 PDE (Porous Medium):

$$u_t = \frac{\nu}{2}(u^2)_{xx}, \quad x \in [0, 1]$$

- 2 IC & BC: Gaussian pulse ($u(x, 0) = 0.1 + \exp(-100(x - 0.5)^2)$); Periodic
- 3 Quadrature: Radau-Right (3 nodes)
- 4 Basic Approximator: Implicit Euler
- 5 Time Parameters: Step size $\Delta t = 0.01$, End time $T_{end} = 0.02$
- 6 Comparisons: Implicit Euler (1 sweep) vs. SDC (4 sweeps) vs. Reference ($\Delta t/10$)

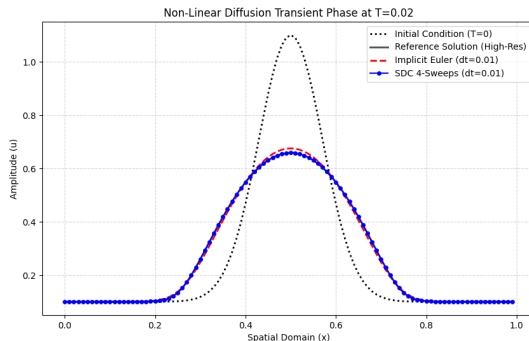
PDE Experiments

Results:

During the active transient phase ($T = 0.02$), the 1st-order Implicit Euler step lags behind, while SDC follows the high-resolution reference solution well. At $T = 0.1$, when diffusion is complete, the error difference between Euler and SDC is minimal; the difference is seen only during the transient phase

Conclusion:

Non-linear equations can be approximated more accurately by using SDC, along with root-finding methods to resolve the non-linear matrix arising from the spatial discretization,



Experiment 2.6: Efficient Shock Modeling via IMEX Splitting

Aim: To demonstrate how IMEX SDC efficiently resolves steep viscous shocks without the computational overhead of non-linear root-finding or severe CFL time-step restrictions.

Setup:

- 1 PDE (Viscous Burgers'):

$$u_t + uu_x = \nu u_{xx}, \quad \nu = 0.02$$

- 2 Scheme Split: Explicit Advection ($-uu_x$) and Implicit Diffusion (νu_{xx})
- 3 Initial Condition: $u(x, 0) = \sin(2\pi x)$
- 4 Quadrature: Radau-Right (3 nodes)
- 5 Sweeper: IMEX 1st-Order Base + 4 SDC Sweeps
- 6 Time Parameters: $\Delta t = 0.005$, $T_{end} = 0.25$ (Maximum shock steepness)

IMEX Splitting:

Instead of treating the entire Partial Differential Equation $u_t = F(u)$ with a single time-integration method, we split the right-hand side into two distinct components:

$$u_t = f_{expl}(u) + f_{impl}(u) = -uu_x + \nu u_{xx}$$

Comments:

- 1 The diffusion term (νu_{xx}) is highly stiff. Treating it explicitly would require a microscopic time step to maintain mathematical stability. The advection term (uu_x) is non-linear. Treating it fully implicitly requires costly Newton-Raphson root-finding at every node.
- 2 By splitting the physics, we process stiff diffusion implicitly (single matrix solve, assured stability) and non-linear advection explicitly (direct calculation). Spectral Deferred Correction then dynamically couples the two together to preserve high-order accuracy.

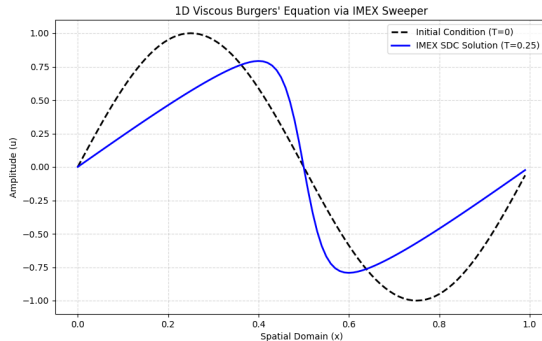
PDE Experiments

Results:

At $T = 0.25$, the initial sine wave forms a steep viscous shock. The IMEX SDC solver captures this cliff perfectly smoothly, without numerical oscillations.

Conclusion:

IMEX SDC provides an optimized, accurate framework for advection-diffusion problems.



Future Roadmap

Here are the plans for the next phase of our project:

① Theoretical Approaches:

- ① Using different interpolating functions (Divided Difference, Hermite)
- ② Stability regime analysis and comparison with RK methods
- ③ Analysis of numerical properties of errors (if possible)

② Further experiments:

- ① Impact of using higher-order base integrators (RK2 and higher-order)
- ② Impact of different collocation nodes (Gauss-Chebyshev, Equispaced)
- ③ Adaptive Stepping control
- ④ Stability-accuracy region plots

Using different interpolating functions

Consider the 1D Heat Equation:

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$$

Using a second-order central difference for the spatial derivative on a grid with spacing Δx :

$$\frac{du_i}{dt} = \frac{\alpha}{(\Delta x)^2} (u_{i-1} - 2u_i + u_{i+1})$$

Formation of ODE

$$\mathbf{u}'(t) = \mathbf{A}\mathbf{u}(t)$$

where $\mathbf{A}$ is the tridiagonal differentiation matrix representing the diffusion operator.

Divided Difference as interpolant

The integral form for the interval $[t_n, t_{n+1}]$, we get

$$\mathbf{u}(t) = \mathbf{u}_n + \int_{t_n}^t \mathbf{A}\mathbf{u}(s)ds$$

Using **Divided Differences**, we approximate $\mathbf{f}(s, \mathbf{u}) = \mathbf{A}\mathbf{u}(s)$

$$\mathbf{f}(s, \mathbf{u}) \approx \mathbf{a}_1 + \mathbf{a}_2(s - t_1) + \mathbf{a}_3(s - t_1)(s - t_2) + \dots$$

where the coefficients $\mathbf{a}_k$ are the divided differences of the vector $\mathbf{A}\mathbf{u}$.

Divided Difference as interpolant

Since the heat equation is stiff, we apply the **Implicit SDC Sweep**:

$$[\mathbf{u}_{m+1}^{k+1} = \mathbf{u}_m^{k+1} + \Delta t_m [\mathbf{A} \mathbf{u}_{m+1}^{k+1} - \mathbf{A} \mathbf{u}_{m+1}^k] + \sum_{k=1}^M \mathbf{a}_k^k S_{m,k}]$$

To find $\mathbf{u}_{m+1}^{k+1}$, we solve the linear system:

$$(\mathbf{I} - \Delta t_m \mathbf{A}) \mathbf{u}_{m+1}^{k+1} = \mathbf{u}_m^{k+1} - \Delta t_m \mathbf{A} \mathbf{u}_{m+1}^k + \sum_{k=1}^M \mathbf{a}_k^k S_{m,k}$$

where $S_{m,k} = \int_{t_m}^{t_{m+1}} \prod_{j=1}^{k-1} (s - t_j) ds$.

Thank You!

Any questions?